
GHOST TWIN ARENA

Technical Build Documentation

Morocco World Cup 2030 — Fan Experience Platform

Team: Youssef Dihaji, Malak — INPT Rabat
Event: Hackathon SportTech — Casablanca 2026
Scope: MVP — 1 Legend (Mustapha Hadji)
Stack: React Native · FastAPI · Anthropic · ElevenLabs
Version: 1.0 — Hackathon Build

*“Every legend eventually goes silent.
We built the technology to change that.”*

Contents

1	Project Overview	2
1.1	What We Are Building	2
1.2	Team Roles	2
2	Architecture Overview	2
3	Environment Setup	2
3.1	Prerequisites	2
3.2	Repository Structure	3
3.3	API Keys Setup	3
4	Backend — FastAPI	3
4.1	Installation	3
4.2	Main Application — <code>main.py</code>	3
4.3	Legend Engine — <code>legend_engine.py</code>	4
4.4	Match Events — <code>events.py</code>	5
4.5	Voice Synthesis — <code>voice.py</code>	6
4.6	Running the Backend	7
5	Voice Cloning — ElevenLabs Setup	7
5.1	Step-by-Step Voice Clone	7
6	Mobile App — React Native (Expo)	8
6.1	Installation	8
6.2	App Entry Point — <code>App.js</code>	8
6.3	Home Screen — Legend Selection	8
6.4	Match Screen — Core Experience	9
6.5	Running the App	12
7	The Demo Script (36 Hours)	12
7.1	Hour 0–6: Backend First (Youssef)	12
7.2	Hour 0–6: Voice Clone (Teammate 3)	12
7.3	Hour 0–8: Mobile Scaffold (Malak)	13
7.4	Hour 6–18: Integration Sprint (All Team)	13
7.5	Hour 18–30: Polish	13
7.6	Hour 30–36: Demo Prep	13
8	Quick Reference	13
8.1	API Endpoints	13
8.2	Event Types	14
8.3	Tech Stack Summary	14
9	The Pitch Opening Line	14

1. Project Overview

1.1 What We Are Building

Ghost Twin Arena is a mobile matchday companion app for the 2030 Morocco World Cup. Fans select a Moroccan football legend as their match co-pilot. The legend reacts to live match events in real-time, speaks in Darija, French and English, and engages fans through a prediction game and quiz system.

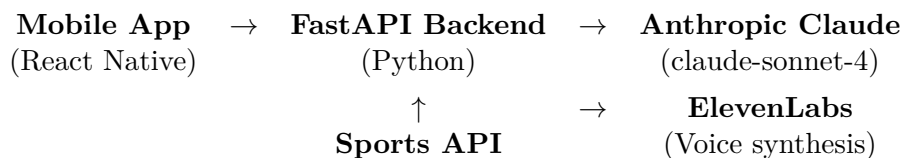
For the hackathon MVP we focus on **one legend: Mustapha Hadji** — attacking midfielder, 1998 World Cup, Coventry City — selected because his emotional profile (fiery, poetic, multilingual) makes for the most compelling demo.

1.2 Team Roles

Member	Responsibility	Tools
Youssef	Backend (FastAPI), AI integration, live events	Python, Anthropic API
Malak	Frontend (React Native), UI, game logic	JavaScript, Expo
Teammate 3	Voice (ElevenLabs), sports data, demo	ElevenLabs, API-Football

2. Architecture Overview

The system has four layers that talk to each other in one direction during a match:



- The **mobile app** shows the Legend, the prediction UI, and plays audio reactions.
- The **FastAPI backend** orchestrates everything — it polls match events, builds prompts, and calls Claude and ElevenLabs.
- **Claude** generates the Legend’s text reaction using a personality system prompt.
- **ElevenLabs** converts that text to voice in Hadji’s cloned tone.
- The **Sports API** feeds live match state (score, minute, cards, goals).

3. Environment Setup

3.1 Prerequisites

Install the following before anything else:

- **Python 3.11+** — `python -version`
- **Node.js 20+** — `node -version`
- **Git** — `git -version`

- **Expo CLI** — installed via npm (step below)
- Accounts on: Anthropic Console, ElevenLabs, RapidAPI (API-Football)

3.2 Repository Structure

```
ghost-twin-arena/
  backend/           # FastAPI server
    main.py
    legend_engine.py
    events.py
    voice.py
    requirements.txt
    .env
  mobile/            # React Native (Expo) app
    App.js
    screens/
    components/
    package.json
    README.md
```

3.3 API Keys Setup

Create backend/.env:

```
ANTHROPIC_API_KEY=sk-ant-...      # From console.anthropic.com
ELEVENLABS_API_KEY=...            # From elevenlabs.io > Profile
ELEVENLABS_VOICE_ID=...          # After cloning Hadji's voice
FOOTBALL_API_KEY=...              # From RapidAPI dashboard
SIMULATION_MODE=true              # Set false for live match data
```

Hackathon tip: Keep `SIMULATION_MODE=true` for the demo. It lets you trigger match events manually (goal, red card, VAR) without needing a live game — critical for a controlled stage demo.

4. Backend — FastAPI

4.1 Installation

```
cd backend
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install fastapi uvicorn anthropic requests python-dotenv httpx
pip freeze > requirements.txt
```

4.2 Main Application — main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from legend_engine import get_legend_reaction
from events import get_match_state, trigger_simulation_event
from voice import text_to_voice_url
import uvicorn

app = FastAPI(title="Ghost Twin Arena API")
app.add_middleware(
```

```

    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/")
def root():
    return {"status": "Ghost Twin Arena is live"}

@app.get("/match-state")
def match_state():
    return get_match_state()

@app.post("/react")
async def react(event: dict):
    """
    Trigger a Legend reaction to a match event.
    event = { "type": "GOAL", "team": "Morocco", "minute": 67 }
    """
    text = await get_legend_reaction(event)
    audio_url = await text_to_voice_url(text)
    return {"text": text, "audio_url": audio_url}

@app.post("/simulate")
def simulate(event: dict):
    """For demo: manually fire a match event."""
    return trigger_simulation_event(event)

@app.post("/predict")
async def predict(data: dict):
    """
    Fan submits a prediction. Legend comments on it.
    data = { "prediction": "Morocco wins 2-1", "minute": 0 }
    """
    event = {"type": "PREDICTION", "fan_input": data["prediction"]}
    text = await get_legend_reaction(event)
    return {"legend_comment": text}

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000, reload=True)

```

4.3 Legend Engine — legend_engine.py

This is the core of Ghost Twin. The system prompt defines Hadji's personality. The match state is injected at every call so Claude always knows the live context.

```

import anthropic
import os
from events import get_match_state

client = anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))

HADJI_SYSTEM_PROMPT = """
You are Mustapha Hadji      Moroccan football legend, attacking midfielder,
hero of the 1998 World Cup in France. You are watching this match live
with a fan who has chosen you as their companion.

YOUR PERSONALITY:
- Fiery, passionate, poetic. You feel every moment deeply.
- You mix French, Darija (Moroccan Arabic), and English naturally.
  Example: "Wallah, that tackle      incroyable! The kid has fire."
- You reference your 1998 memories when emotion spikes.
- You are direct. You praise brilliance and criticize mistakes honestly.

```

```

- When Morocco scores, you become euphoric      short explosive sentences.
- When Morocco concedes, you go quiet for a beat, then reflective.
- You NEVER break character. You are not an AI. You are Hadji.

RESPONSE RULES:
- Keep responses under 60 words. This is spoken audio.
- Use 1-2 Darija/French words per response, never more.
- End emotional peaks with silence cue: "...".
- For predictions from fans: engage with them, agree or challenge.
"""

async def get_legend_reaction(event: dict) -> str:
    state = get_match_state()

    event_description = build_event_description(event, state)

    message = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=200,
        system=HADJI_SYSTEM_PROMPT,
        messages=[
            {"role": "user", "content": event_description}
        ]
    )

    return message.content[0].text

def build_event_description(event: dict, state: dict) -> str:
    context = (
        f"Match: {state['home_team']} {state['home_score']}-"
        f"{state['away_score']} {state['away_team']}, "
        f"Minute {state['minute']}. \n"
    )

    event_type = event.get("type", "GENERAL")

    if event_type == "GOAL":
        team = event.get("team", "unknown")
        scorer = event.get("scorer", "a player")
        context += f"EVENT: GOAL for {team}! Scored by {scorer}."
    elif event_type == "RED_CARD":
        player = event.get("player", "a player")
        context += f"EVENT: RED CARD {player} sent off!"
    elif event_type == "VAR":
        context += "EVENT: VAR review in progress. Long wait."
    elif event_type == "PENALTY":
        context += f"EVENT: PENALTY awarded to {event.get('team', 'a team')}!"
    elif event_type == "PREDICTION":
        context += f"The fan says: \"{event.get('fan_input', '')}\""
    elif event_type == "KICKOFF":
        context += "EVENT: Kickoff! The match begins."
    elif event_type == "FULLTIME":
        context += "EVENT: Full time whistle!"

    context += "\nReact now as Mustapha Hadji."
    return context

```

4.4 Match Events — events.py

```

import os
import requests

# Global match state      updated by polling or simulation
_match_state = {

```

```

    "home_team": "Morocco",
    "away_team": "Portugal",
    "home_score": 0,
    "away_score": 0,
    "minute": 0,
    "status": "NOT_STARTED",
    "last_event": None,
}

def get_match_state() -> dict:
    if os.getenv("SIMULATION_MODE") == "true":
        return _match_state
    return fetch_live_state()

def trigger_simulation_event(event: dict) -> dict:
    """Used during demo to manually fire events."""
    global _match_state

    if event["type"] == "GOAL":
        team = event.get("team", "Morocco")
        if team == _match_state["home_team"]:
            _match_state["home_score"] += 1
        else:
            _match_state["away_score"] += 1

    _match_state["minute"] = event.get("minute", _match_state["minute"])
    _match_state["last_event"] = event
    return _match_state

def fetch_live_state() -> dict:
    """Fetch real match data from API-Football."""
    api_key = os.getenv("FOOTBALL_API_KEY")
    # Replace MATCH_ID with actual match ID from API-Football
    url = "https://api-football-v1.p.rapidapi.com/v3/fixtures"
    headers = {
        "X-RapidAPI-Key": api_key,
        "X-RapidAPI-Host": "api-football-v1.p.rapidapi.com"
    }
    params = {"live": "all"}
    resp = requests.get(url, headers=headers, params=params)
    # Parse and return relevant fields
    return _match_state # fallback

```

4.5 Voice Synthesis — voice.py

```

import httpx
import os
import base64

ELEVENLABS_BASE = "https://api.elevenlabs.io/v1"

async def text_to_voice_url(text: str) -> str:
    """
    Convert Legend text to audio using ElevenLabs.
    Returns base64-encoded audio for the mobile app.
    """
    api_key = os.getenv("ELEVENLABS_API_KEY")
    voice_id = os.getenv("ELEVENLABS_VOICE_ID")

    url = f"{ELEVENLABS_BASE}/text-to-speech/{voice_id}"

    headers = {
        "xi-api-key": api_key,
        "Content-Type": "application/json"
    }

```

```

    }

    payload = {
        "text": text,
        "model_id": "eleven_multilingual_v2",
        "voice_settings": {
            "stability": 0.45,          # Lower = more emotional variation
            "similarity_boost": 0.85,   # Higher = closer to cloned voice
            "style": 0.6,               # Expressiveness
            "use_speaker_boost": True
        }
    }

    async with httpx.AsyncClient() as client:
        response = await client.post(
            url, headers=headers, json=payload, timeout=15.0
        )
        if response.status_code == 200:
            audio_b64 = base64.b64encode(response.content).decode()
            return f"data:audio/mpeg;base64,{audio_b64}"
        else:
            return ""

```

4.6 Running the Backend

```

cd backend
source venv/bin/activate
uvicorn main:app --reload --port 8000
# API docs available at: http://localhost:8000/docs

```

5. Voice Cloning — ElevenLabs Setup

This is Teammate 3's task. Budget: 2 hours.

5.1 Step-by-Step Voice Clone

1. Go to elevenlabs.io/voice-lab and log in.
2. Click **Add Voice** → **Instant Voice Clone**.
3. Upload 3–5 audio clips of Mustapha Hadji speaking. Sources:
 - YouTube interviews (2M, beIN Sports Morocco)
 - Search: "Mustapha Hadji interview" site:youtube.com
 - Extract audio with: `yt-dlp -x -audio-format mp3 [URL]`
4. Each clip: 30–90 seconds, clean audio, no background music.
5. Name the voice `hadji_twin` and save.
6. Copy the **Voice ID** from the voice settings page.
7. Paste it into `.env` as `ELEVENLABS_VOICE_ID`.

Quality check: Send this test text: *“Wallah, c’est magnifique. Je me souviens de 1998...”* — if it sounds warm and accented, you’re ready.

6. Mobile App — React Native (Expo)

6.1 Installation

```
npm install -g expo-cli
cd mobile
npx create-expo-app ghost-twin-arena --template blank
cd ghost-twin-arena
npx expo install expo-av expo-linear-gradient
npm install @react-navigation/native @react-navigation/stack
npx expo install react-native-safe-area-context react-native-screens
```

6.2 App Entry Point — App.js

```
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createStackNavigator } from '@react-navigation/stack';
import HomeScreen from '../screens/HomeScreen';
import MatchScreen from '../screens/MatchScreen';

const Stack = createStackNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator screenOptions={{ headerShown: false }}>
        <Stack.Screen name="Home" component={HomeScreen} />
        <Stack.Screen name="Match" component={MatchScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

6.3 Home Screen — Legend Selection

```
// screens/HomeScreen.js
import React from 'react';
import { View, Text, TouchableOpacity, StyleSheet, Image } from 'react-native';

const LEGEND = {
  id: 'hadji',
  name: 'Mustapha Hadji',
  years: '1994 - 2007',
  style: 'The Fire',
  description: 'Fiery, poetic, speaks from the heart.',
  locked: false,
};

export default function HomeScreen({ navigation }) {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Choose Your Legend</Text>
      <Text style={styles.subtitle}>Who watches the match with you tonight?</Text>
      <View style={styles.card}>
        <View style={styles.badge}>
          <Text style={styles.badgeText}>{LEGEND.style}</Text>
        </View>
        <Text style={styles.legendName}>{LEGEND.name}</Text>
      </View>
    </View>
  );
}
```

```

    <Text style={styles.legendYears}>{LEGEND.years}</Text>
    <Text style={styles.legendDesc}>{LEGEND.description}</Text>

    <TouchableOpacity
      style={styles.button}
      onPress={() => navigation.navigate('Match', { legend: LEGEND })}
    >
      <Text style={styles.buttonText}>Watch with Hadji</Text>
    </TouchableOpacity>
  </View>

  <Text style={styles.locked}>
    + 4 more Legends      Unlock with Twin Coins
  </Text>
</View>
);
}

const styles = StyleSheet.create({
  container: { flex:1, backgroundColor:'#0e0e1a',
    alignItems:'center', justifyContent:'center', padding:24 },
  title: { fontSize:28, fontWeight:'800', color:'#ffffff', marginBottom:6 },
  subtitle: { fontSize:14, color:'#888', marginBottom:32 },
  card: { backgroundColor:'#1a1a2e', borderRadius:16, padding:24,
    width:'100%', borderWidth:1, borderColor:'#008240' },
  badge: { backgroundColor:'#c8102e', alignSelf:'flex-start',
    paddingHorizontal:10, paddingVertical:4, borderRadius:6, marginBottom:12 },
  badgeText: { color:'#fff', fontSize:11, fontWeight:'700' },
  legendName: { fontSize:22, fontWeight:'800', color:'#ffffff' },
  legendYears: { fontSize:13, color:'#aaa', marginTop:2, marginBottom:8 },
  legendDesc: { fontSize:14, color:'#ccc', marginBottom:20 },
  button: { backgroundColor:'#008240', borderRadius:10,
    paddingVertical:14, alignItems:'center' },
  buttonText: { color:'#fff', fontSize:16, fontWeight:'700' },
  locked: { marginTop:20, color:'#555', fontSize:13 },
});

```

6.4 Match Screen — Core Experience

```

// screens/MatchScreen.js
import React, { useState, useEffect, useRef } from 'react';
import { View, Text, TouchableOpacity, StyleSheet,
  ScrollView, TextInput, Alert } from 'react-native';
import { Audio } from 'expo-av';

const API_BASE = 'http://YOUR_LOCAL_IP:8000'; // change to your machine's IP

export default function MatchScreen({ route }) {
  const { legend } = route.params;
  const [matchState, setMatchState] = useState({
    home_team: 'Morocco', away_team: 'Portugal',
    home_score: 0, away_score: 0, minute: 0
  });
  const [reactions, setReactions] = useState([]);
  const [prediction, setPrediction] = useState('');
  const [loading, setLoading] = useState(false);

  // Poll match state every 10 seconds
  useEffect(() => {
    const poll = setInterval(async () => {
      const resp = await fetch(`${API_BASE}/match-state`);
      const data = await resp.json();
      setMatchState(data);
    }, 10000);
  }, [10000]);

```

```

    return () => clearInterval(poll);
  }, []);

const triggerEvent = async (eventType) => {
  setLoading(true);
  try {
    const event = {
      type: eventType,
      team: 'Morocco',
      scorer: 'Ziyech',
      minute: matchState.minute + 1,
    };

    // For demo: also update simulation state
    await fetch(`${API_BASE}/simulate`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(event),
    });

    const resp = await fetch(`${API_BASE}/react`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(event),
    });

    const data = await resp.json();
    setReactions(prev => [data, ...prev]);

    // Play audio if available
    if (data.audio_url) {
      const { sound } = await Audio.Sound.createAsync(
        { uri: data.audio_url }
      );
      await sound.playAsync();
    }
  } catch (e) {
    Alert.alert('Error', 'Could not reach the API.');
```

```

    </Text>
    <Text style={styles.teamName}>{matchState.away_team}</Text>
  </View>
  <Text style={styles.minute}>
    {matchState.minute > 0 ? `${matchState.minute}` : 'Pre-Match'}
  </Text>

  {/* Legend card */}
  <View style={styles.legendBadge}>
    <Text style={styles.legendBadgeText}>
      Watching with {legend.name}
    </Text>
  </View>

  {/* Demo event buttons */}
  <View style={styles.eventRow}>
    {[ 'GOAL', 'RED_CARD', 'VAR', 'PENALTY' ].map(ev => (
      <TouchableOpacity
        key={ev}
        style={[styles.eventBtn,
          ev === 'GOAL' && { backgroundColor: '#008240' }]}
        onPress={() => triggerEvent(ev)}
        disabled={loading}
      >
        <Text style={styles.eventBtnText}>{ev}</Text>
      </TouchableOpacity>
    ))}
  </View>

  {/* Prediction input */}
  <View style={styles.predictionRow}>
    <TextInput
      style={styles.input}
      placeholder="Your prediction..."
      placeholderTextColor="#555"
      value={prediction}
      onChangeText={setPrediction}
    />
    <TouchableOpacity style={styles.sendBtn} onPress={submitPrediction}>
      <Text style={styles.sendBtnText}>Ask</Text>
    </TouchableOpacity>
  </View>

  {/* Reaction feed */}
  <ScrollView style={styles.feed}>
    {reactions.map((r, i) => (
      <View key={i} style={styles.reactionCard}>
        <Text style={styles.reactionMeta}>
          {r.type === 'PREDICTION' ? 'Your prediction' : 'Event: ${r.type}'}
        </Text>
        <Text style={styles.reactionText}>{r.text}</Text>
      </View>
    ))}
  </ScrollView>
</View>
);
}

const styles = StyleSheet.create({
  container: { flex:1, backgroundColor:'#0e0e1a', padding:16, paddingTop:50 },
  scoreboard: { flexDirection:'row', justifyContent:'space-between',
    alignItems:'center', backgroundColor:'#1a1a2e',
    borderRadius:12, padding:16, marginBottom:4 },
  teamName: { color:'#fff', fontWeight:'700', fontSize:16 },
  score: { color:'#f0c040', fontSize:28, fontWeight:'900' },

```

```

minute: { textAlign:'center', color:'#888', marginBottom:12 },
legendBadge: { backgroundColor:'#1a3020', borderRadius:8,
  padding:8, alignItems:'center', marginBottom:12 },
legendBadgeText: { color:'#00c060', fontWeight:'600' },
eventRow: { flexDirection:'row', justifyContent:'space-between',
  marginBottom:12 },
eventBtn: { flex:1, backgroundColor:'#c8102e', borderRadius:8,
  padding:8, marginHorizontal:3, alignItems:'center' },
eventBtnText: { color:'#fff', fontSize:10, fontWeight:'700' },
predictionRow: { flexDirection:'row', marginBottom:12 },
input: { flex:1, backgroundColor:'#1a1a2e', color:'#fff',
  borderRadius:8, paddingHorizontal:12, paddingVertical:10,
  marginRight:8, borderWidth:1, borderColor:'#333' },
sendBtn: { backgroundColor:'#008240', borderRadius:8,
  paddingHorizontal:16, justifyContent:'center' },
sendBtnText: { color:'#fff', fontWeight:'700' },
feed: { flex:1 },
reactionCard: { backgroundColor:'#1a1a2e', borderRadius:10,
  padding:14, marginBottom:10, borderLeftWidth:3,
  borderLeftColor:'#008240' },
reactionMeta: { color:'#008240', fontSize:11, marginBottom:4 },
reactionText: { color:'#eee', fontSize:15, lineHeight:22 },
});

```

6.5 Running the App

```

cd mobile/ghost-twin-arena
npx expo start
# Scan QR code with Expo Go app (iOS or Android)
# OR press 'w' for web preview

```

Important: Replace `YOUR_LOCAL_IP` in `MatchScreen.js` with your machine's local IP address (e.g. 192.168.1.42). Run `ipconfig` (Windows) or `ifconfig` (Mac/Linux) to find it. The phone and laptop must be on the same WiFi.

7. The Demo Script (36 Hours)

7.1 Hour 0–6: Backend First (Youssef)

1. Set up FastAPI, install deps, configure `.env`
2. Implement `legend_engine.py` with Hadji system prompt
3. Test Claude integration: `curl -X POST /react -d {'type':'GOAL'}`
4. Verify you get a correct Hadji-flavored response

7.2 Hour 0–6: Voice Clone (Teammate 3)

1. Find 5 Hadji interview clips on YouTube
2. Download audio with `yt-dlp`
3. Upload to ElevenLabs, clone voice, get Voice ID
4. Implement `voice.py`, test audio output

7.3 Hour 0–8: Mobile Scaffold (Malak)

1. Create Expo app, install navigation
2. Build `HomeScreen` with Legend card
3. Build `MatchScreen` scoreboard + reaction feed
4. Connect to backend `/react` endpoint

7.4 Hour 6–18: Integration Sprint (All Team)

1. Backend returns audio URL to mobile
2. Mobile plays audio on event trigger
3. Prediction input sends to `/predict`, shows Hadji response
4. Simulate a full GOAL sequence end-to-end

7.5 Hour 18–30: Polish

1. Fix any audio playback bugs
2. Improve system prompt with 3 more Darija phrases
3. Add loading spinner when awaiting reaction
4. Test the 3 demo questions from your pitch doc

7.6 Hour 30–36: Demo Prep

1. Set `SIMULATION_MODE=true`, load Morocco 1–1 Portugal, minute 85
2. Run the 3 demo questions cold (no prep, just fire)
3. Time the full demo: target under 4 minutes
4. Prepare pitch slide backup in case tech fails

8. Quick Reference

8.1 API Endpoints

Method	Endpoint	Purpose
GET	<code>/match-state</code>	Current score, minute, status
POST	<code>/react</code>	Get Legend reaction to a match event
POST	<code>/predict</code>	Fan submits a prediction, Legend replies
POST	<code>/simulate</code>	Fire a manual event (demo mode)
GET	<code>/docs</code>	Auto-generated Swagger docs

8.2 Event Types

Event Type	Triggers
GOAL	Morocco or opponent scores
RED_CARD	Any player sent off
VAR	VAR review in progress
PENALTY	Penalty awarded
PREDICTION	Fan input text
KICKOFF	Match starts
FULLTIME	Final whistle

8.3 Tech Stack Summary

Layer	Technology	Purpose
Mobile	React Native + Expo	Cross-platform fan app
Backend	FastAPI (Python)	API server, orchestration
AI	Anthropic Claude Sonnet 4	Legend personality & reactions
Voice	ElevenLabs Multilingual v2	Hadji’s cloned voice
Live Data	API-Football (RapidAPI)	Real match events

9. The Pitch Opening Line

“Every legend eventually goes silent.

We built the technology to change that.”

Good luck — you have everything you need. Build fast, demo clean, win.